

1. Write a C program for GRAPH

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define MAX_VERTICES 100
4 void createGraph(int graph[MAX_VERTICES][MAX_VERTICES], int vertices) {
5     for (int i = 0; i < vertices; i++) {
6         for (int j = 0; j < vertices; j++) {
7             graph[i][j] = 0;
8         }
9     }
10 }
11 void addEdge(int graph[MAX_VERTICES][MAX_VERTICES], int u, int v) {
12     graph[u][v] = 1;
13     graph[v][u] = 1;
14 }
15 void printGraph(int graph[MAX_VERTICES][MAX_VERTICES], int vertices) {
16     printf("Adjacency Matrix:\n");
17     for (int i = 0; i < vertices; i++) {
18         for (int j = 0; j < vertices; j++) {
19             printf("%d ", graph[i][j]);
20         }
21         printf("\n");
22     }
23 }
24 int main() {
25     int vertices = 5;
26     int graph[MAX_VERTICES][MAX_VERTICES];
27     createGraph(graph, vertices);
28     addEdge(graph, 0, 1);
29     addEdge(graph, 0, 4);
30     addEdge(graph, 1, 2);
31     addEdge(graph, 1, 3);
32     addEdge(graph, 1, 4);
33     addEdge(graph, 2, 3);
34     addEdge(graph, 3, 4);
35     printGraph(graph, vertices);
36     return 0;
37 }
```

```
/tmp/6Lzpp1oU8.o
Adjacency Matrix:
0 1 0 0 1
1 0 1 1 1
0 1 0 1 0
0 1 1 0 1
1 1 0 1 0

=== Code Execution Successful ===
```

2. Write a C program for TOPOLOGICAL GRAPH.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define MAX 100
4 struct Node {
5     int vertex;
6     struct Node* next;
7 };
8 struct Graph {
9     int numVertices;
10    struct Node** adjLists;
11    int* visited;
12 };
13 struct Node* createNode(int v) {
14     struct Node* newNode = malloc(sizeof(struct Node));
15     newNode->vertex = v;
16     newNode->next = NULL;
17     return newNode;
18 }
19 struct Graph* createGraph(int vertices) {
20     struct Graph* graph = malloc(sizeof(struct Graph));
21     graph->numVertices = vertices;
22     graph->adjLists = malloc(vertices * sizeof(struct Node*));
23     graph->visited = malloc(vertices * sizeof(int));
24     for (int i = 0; i < vertices; i++) {
25         graph->adjLists[i] = NULL;
26         graph->visited[i] = 0;
27     }
28     return graph;
29 }
30 void addEdge(struct Graph* graph, int src, int dest) {
31     struct Node* newNode = createNode(dest);
32     newNode->next = graph->adjLists[src];
33     graph->adjLists[src] = newNode;
34 }
```

```
/tmp/QV63GWzxUH.o
Topological Sort Order:
5 4 2 3 1 0

=== Code Execution Successful ===
```

```

34 }
35 void topologicalSortUtil(struct Graph* graph, int v, int* stack, int* top) {
36     graph->visited[v] = 1;
37     struct Node* adjList = graph->adjLists[v];
38     struct Node* temp = adjList;
39     while (temp != NULL) {
40         int connectedVertex = temp->vertex;
41         if (!graph->visited[connectedVertex]) {
42             topologicalSortUtil(graph, connectedVertex, stack, top);
43         }
44         temp = temp->next;
45     }
46     stack[(*top)++] = v;
47 }
48 void topologicalSort(struct Graph* graph) {
49     int* stack = malloc(graph->numVertices * sizeof(int));
50     int top = 0;
51     for (int i = 0; i < graph->numVertices; i++) {
52         if (!graph->visited[i]) {
53             topologicalSortUtil(graph, i, stack, &top);
54         }
55     }
56     printf("Topological Sort Order:\n");
57     for (int i = top - 1; i >= 0; i--) {
58         printf("%d ", stack[i]);
59     }
60     printf("\n");
61     free(stack);
62 }
63 int main() {
64     int vertices = 6;
65     struct Graph* graph = createGraph(vertices);
66     addEdge(graph, 5, 2);
67 }

```

```

/tmp/QV63GwzxUH.o
Topological Sort Order:
5 4 2 3 1 0

=== Code Execution Successful ===

```

```

63 }
64 int main() {
65     int vertices = 6;
66     struct Graph* graph = createGraph(vertices);
67     addEdge(graph, 5, 2);
68     addEdge(graph, 5, 0);
69     addEdge(graph, 4, 0);
70     addEdge(graph, 4, 1);
71     addEdge(graph, 2, 3);
72     addEdge(graph, 3, 1);
73     topologicalSort(graph);
74     return 0;
75 }

```